

Task 3

Longest Balanced Substring

Language: Python

Algorithm 1 — (Non-Recursive)

PSEUDOCODE

```
FUNCTION my_len(s)
    count <- 0
    FOR each character in s DO
        count <- count + 1
    END FOR
    RETURN count
END FUNCTION

FUNCTION longest_balanced(s)
    n <- my_len(s)
    max_len <- 0
    i <- 0
    WHILE i < n DO
        freq <- empty dictionary
        j <- i
        WHILE j < n DO
            c <- s[j]
            IF c exists in freq THEN
                freq[c] <- freq[c] + 1
            ELSE
                freq[c] <- 1
            END IF
            key_count <- 0
            FOR each key in freq DO
                key_count <- key_count + 1
            END FOR
            IF key_count > 2 THEN
                BREAK
            END IF
            IF key_count = 2 THEN
                val1 <- 0
                val2 <- 0
                first <- TRUE
                FOR each key in freq DO
                    IF first = TRUE THEN
                        val1 <- freq[key]
                        first <- FALSE
                    ELSE
                        val2 <- freq[key]
                    END IF
                END FOR
                IF val1 = val2 THEN
                    length <- j - i + 1
                    IF length > max_len THEN
                        max_len <- length
                    END IF
                END IF
            END IF
            j <- j + 1
        END WHILE
        i <- i + 1
    END WHILE
    RETURN max_len
END FUNCTION
```

Python Implementation

Python Implementation

```

1  def my_len(s):
2      count = 0
3      for _ in s:
4          count = count + 1
5      return count
6
7
8  def longest_balanced(s):
9      n = my_len(s)
10     max_len = 0
11
12     i = 0
13     while i < n:
14         freq = {}
15
16         j = i
17         while j < n:
18             c = s[j]
19
20             if c in freq:
21                 freq[c] = freq[c] + 1
22             else:
23                 freq[c] = 1
24
25             key_count = 0
26             for _ in freq:
27                 key_count = key_count + 1
28
29             if key_count > 2:
30                 break
31
32             if key_count == 2:
33                 val1 = 0
34                 val2 = 0
35                 first = True
36                 for k in freq:
37                     if first:
38                         val1 = freq[k]
39                         first = False
40                     else:
41                         val2 = freq[k]
42
43                 if val1 == val2:
44                     length = j - i + 1
45                     if length > max_len:
46                         max_len = length
47
48             j = j + 1
49
50         i = i + 1
51
52     return max_len

```

Time Complexity

$O(n^2)$

The algorithm uses two nested loops to generate and check all possible substrings of the given string.

1. Outer Loop

```
i = 0
while i < n:
```

This loop selects the starting index of each substring.

It runs n times, so its complexity is $O(n)$.

2. Inner Loop

```
j = i
while j < n:
```

This loop expands the substring from index i to the end of the string.

In the worst case, it runs:

- n times when $i = 0$
- $n-1$ times when $i = 1$
- $n-2$ times when $i = 2$
- ...

So total iterations are:

$$n + (n-1) + (n-2) + \dots + 1 = n(n+1)/2$$

This simplifies to: $O(n^2)$

Algorithm 2 — Recursive

PSEUDOCODE

```
FUNCTION my_len(s)
  count <- 0
  FOR each character in s DO
    count <- count + 1
  END FOR
  RETURN count
END FUNCTION

FUNCTION longest_balanced_recursive(s, start, best)
  n <- my_len(s)
  IF start >= n - 1 THEN
    RETURN best
  END IF
  freq <- empty dictionary
  end <- start
  WHILE end < n DO
    c <- s[end]
    IF c exists in freq THEN
      freq[c] <- freq[c] + 1
    ELSE
      freq[c] <- 1
    END IF
    key_count <- 0
    FOR each key in freq DO
      key_count <- key_count + 1
    END FOR
    IF key_count > 2 THEN
      BREAK
    END IF
    IF key_count = 2 THEN
      val1 <- 0
      val2 <- 0
      first <- TRUE
      FOR each key in freq DO
        IF first = TRUE THEN
          val1 <- freq[key]
          first <- FALSE
        ELSE
          val2 <- freq[key]
        END IF
      END FOR
      IF val1 = val2 THEN
        length <- end - start + 1
        IF length > best THEN
          best <- length
        END IF
      END IF
    END IF
    end <- end + 1
  END WHILE
  RETURN longest_balanced_recursive(s, start + 1, best)
END FUNCTION
```

Python Implementation

```
1  def my_len(s):
2      count = 0
3      for _ in s:
4          count = count + 1
5      return count
6
7
8  def longest_balanced_recursive(s, start=0, best=0):
9
10     n = my_len(s)
11
12     if start >= n - 1:
13         return best
14
15     freq = {}
16
17     end = start
18     while end < n:
19         c = s[end]
20
21         if c in freq:
22             freq[c] = freq[c] + 1
23         else:
24             freq[c] = 1
25
26         key_count = 0
27         for _ in freq:
28             key_count = key_count + 1
29
30         if key_count > 2:
31             break
32
33         if key_count == 2:
34             val1 = 0
35             val2 = 0
36             first = True
37             for k in freq:
38                 if first:
39                     val1 = freq[k]
40                     first = False
41                 else:
42                     val2 = freq[k]
43
44             if val1 == val2:
45                 length = end - start + 1
46                 if length > best:
47                     best = length
48
49         end = end + 1
50
51     return longest_balanced_recursive(s, start + 1, best)
```

Time Complexity

The algorithm uses a recursive approach where for each starting index (start), it iterates through all possible ending indices (end) to form substrings.

- The outer recursion runs for $O(n)$ starting positions.
- For each position, the inner loop may scan up to $O(n)$ elements.

Therefore, the total number of operations is:

$$n + (n-1) + (n-2) + \dots + 1 = O(n^2)$$

Final Time Complexity:

$$O(n^2)$$

Comparison Between the Two Algorithms

Property	Algorithm 1	Algorithm 2
Type	Non-Recursive	Recursive
Outer iteration	while loop	function calls itself
Inner logic	same	same
Time Complexity	$O(n^2)$	$O(n^2)$
Space Complexity	$O(1)$	$O(n)$ call stack
Base case needed	No	Yes
Stop condition	break on 3rd char	break on 3rd char
Memory usage	Lower	Higher
Code readability	Simple	Moderate

Summary

Time Complexity: Both algorithms have $O(n^2)$ time complexity. The outer loop (or recursive call) runs n times and the inner loop runs up to n times per iteration, giving $n \times n = n^2$ total operations.